

Consultas ORM vs SQL

Nombre: Gabriel Villalobos Aguirre. Grupo: 9°B "Tics" **Fecha:** 05/08/26.

Problema 1: Comentarios creados en un rango de fechas:

1.-Planteamiento de la Consulta:

Obtener y mostrar todos los comentarios guardados en la base de datos cuya fecha de creación se encuentre comprendida entre el 20 de junio y el 4 de agosto.

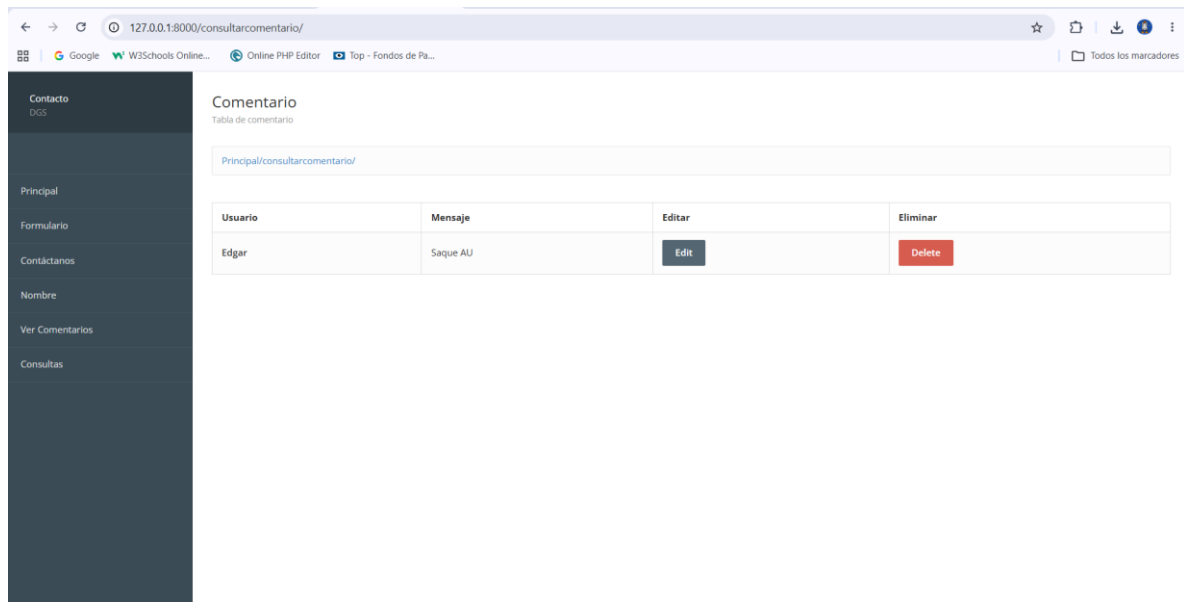
2.-Consulta con Django ORM:

```
def consultarcomentario(request):  
    comentarios = ComentarioContacto.objects.filter(created__range=["2026-06-20", "2026-08-4"])  
    return render(request, "registros/consultaComentarios.html", {'comentarios': comentarios})
```

3.-Consulta equivalente en SQL:

SELECT * FROM registros_comentarioContacto WHERE created BETWEEN '2026-06-20 00:00:00' AND '2026-08-04 23:59:59'

Captura de ejecución:



The screenshot shows a web browser at the address 127.0.0.1:8000/consultarcomentario/. The page has a dark sidebar with navigation links: Contacto DGS, Principal, Formulario, Contactanos, Nombre, Ver Comentarios, and Consultas. The main content area is titled 'Comentario' and 'Tabla de comentario'. It displays a table with one row of data. The table has columns for 'Usuario', 'Mensaje', 'Editar', and 'Eliminar'. The 'Usuario' column contains 'Edgar', and the 'Mensaje' column contains 'Saque AU'. The 'Editar' column has a blue 'Edit' button, and the 'Eliminar' column has a red 'Delete' button.

Usuario	Mensaje	Editar	Eliminar
Edgar	Saque AU	Edit	Delete

Problema 2: Búsqueda de una expresión en el comentario.

1.-Planteamiento de la Consulta:

Filtrar la base de datos para obtener todos los comentarios que contengan una expresión específica dentro del campo de mensaje.

En este caso se buscarán todos los comentarios que contengan la palabra **"Tigres"**.

2.-Consulta con Django ORM:

```
def consultarexpresion(request):  
    comentarios = ComentarioContacto.objects.filter(mensaje__icontains="Tigres")  
    return render(request, "registros/consultaComentarios.html", {"comentarios": comentarios})
```

3.-Consulta equivalente en SQL:

SELECT * FROM registros_comentariocontacto WHERE mensaje LIKE '%Tigres%';

4.-Captura de ejecución:

127.0.0.1:8000/consultarexpresion/

Google W3Schools Online... Online PHP Editor Top - Fondos de Pa... Todos los marcadores

Contacto
Dios

Principal

Formulario

Contactanos

Nombre

Ver Comentarios

Consultas

Comentario
Tabla de comentario

Principal/consultarexpresion/

Usuario	Mensaje	Editar	Eliminar
migue	puro tigres	Edit	Delete
Axel	Arriba el Tigres	Edit	Delete

Problema 3: Comentarios pertenecientes a un usuario específico.

1.-Planteamiento de la Consulta:

Filtrar la base de datos para extraer de forma exclusiva los comentarios registrados por un usuario determinado.

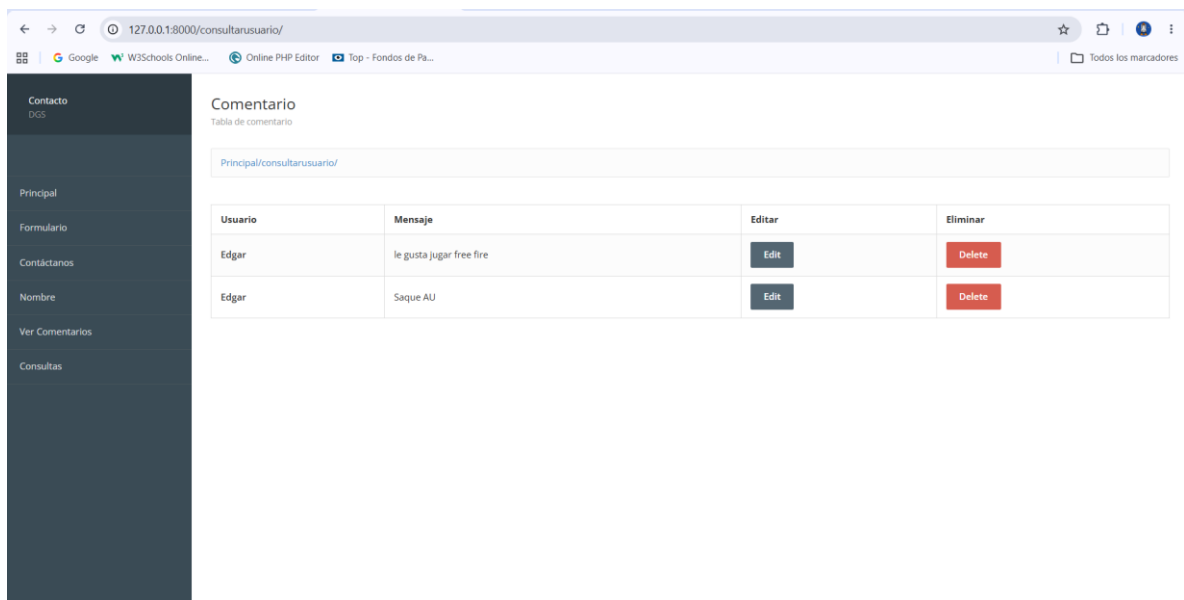
2.-Consulta con Django ORM:

```
def consultarusuario(request):  
    comentarios = ComentarioContacto.objects.filter(usuario="Edgar")  
    return render(request, "registros/consultaComentarios.html", {"comentarios": comentarios})
```

3.-Consulta equivalente en SQL:

```
SELECT * FROM registros_comentariocontacto WHERE usuario = Edgar;
```

4.-Captura de ejecución:



The screenshot shows a web browser at the URL `127.0.0.1:8000/consultarusuario/`. The page has a dark sidebar on the left with navigation links: `Contacto`, `Principal`, `Formulario`, `Contactanos`, `Nombre`, `Ver Comentarios`, and `Consultas`. The main content area is titled `Comentario` with the subtitle `Tabla de comentario`. Below the title is a breadcrumb `Principal/consultarusuario/`. A table displays the comments for the user 'Edgar'.

Usuario	Mensaje	Editar	Eliminar
Edgar	le gusta jugar free fire	<button>Edit</button>	<button>Delete</button>
Edgar	Saque AU	<button>Edit</button>	<button>Delete</button>

4.-Consulta 1 solicitada en la clase anterior

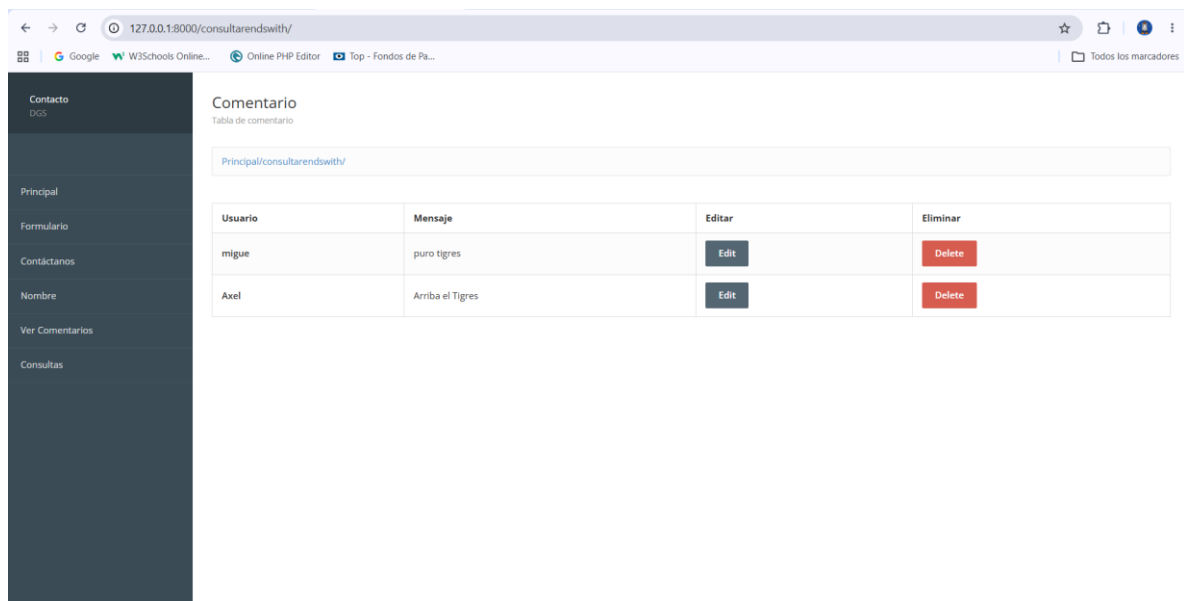
1.-Consulta nueva 1 con Django ORM:.

```
# tarea lunes
def consultarendswith (request):
    # __endswith busca mensajes que finalicen con ese texto
    comentarios = ComentarioContacto.objects.filter(mensaje__endswith="es")
    return render(request, "registros/consultaComentarios.html", {'comentarios': comentarios})
```

2.-Consulta nueva 1 equivalente en SQL:

SELECT * FROM registros_comentariocontacto WHERE mensaje LIKE '%es';

3.-Captura de ejecución de la nueva consulta 1:



The screenshot shows a web browser at the URL `127.0.0.1:8000/consultarendswith/`. The application has a dark sidebar with navigation links: **Contacto** (DGS), **Principal**, **Formulario**, **Contactanos**, **Nombre**, **Ver Comentarios**, and **Consultas**. The main content area is titled **Comentario** with the subtitle **Tabla de comentario**. Below the title is a breadcrumb [Principal/consultarendswith/](#). A table displays the results of the query, showing two records. Each record has columns for **Usuario**, **Mensaje**, **Editar** (with an **Edit** button), and **Eliminar** (with a **Delete** button).

Usuario	Mensaje	Editar	Eliminar
migue	puro tigres	<button>Edit</button>	<button>Delete</button>
Axel	Arriba el Tigres	<button>Edit</button>	<button>Delete</button>

5.-Consulta 2 solicitada en la clase anterior:

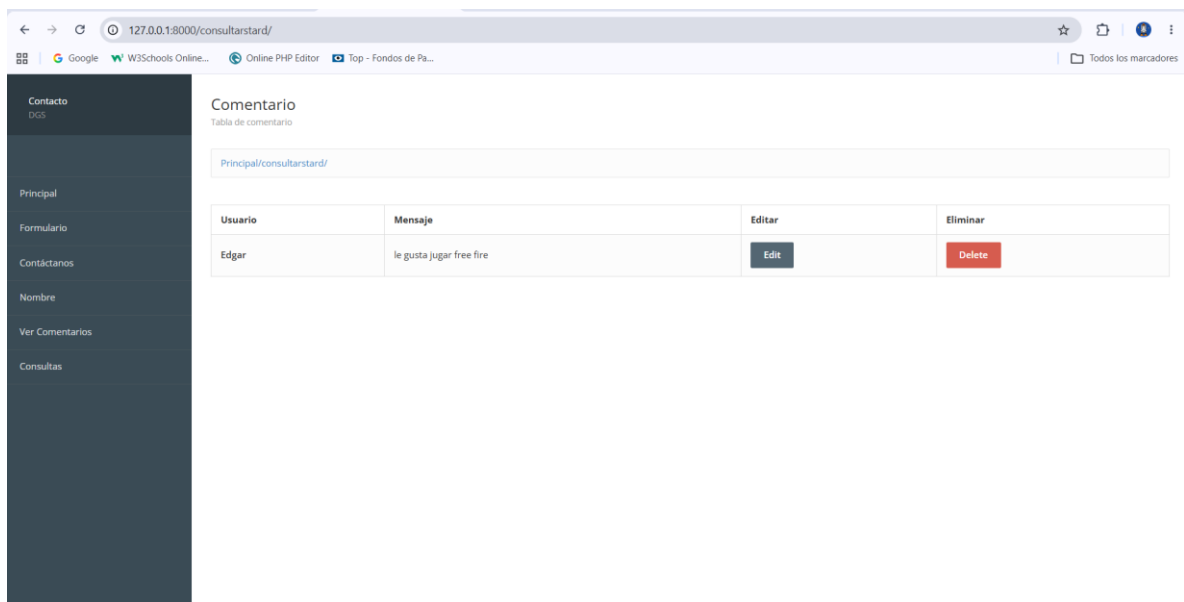
1.-Consulta nueva 2 con Django ORM:

```
# tarea lunes
def consultarstard(request):
    # __startswith busca mensajes que inicien con ese texto
    comentarios = ComentarioContacto.objects.filter(mensaje__startswith="le")
    return render(request, "registros/consultaComentarios.html", {'comentarios': comentarios})
```

2.-Consulta nueva 2 equivalente en SQL.:

SELECT * FROM registros_comentariocontacto WHERE mensaje LIKE '%le';

3.-Captura de ejecución de la nueva consulta 2:



6.-CONCLUSIÓN: VENTAJAS Y DESVENTAJAS DE UN ORM VS SQL:

El uso de ORM (Object Relational Mapping) y SQL permite trabajar con bases de datos, pero cada uno tiene características diferentes. Un ORM facilita la comunicación entre la aplicación y la base de datos, ya que permite realizar consultas utilizando objetos y código del lenguaje de programación en lugar de escribir directamente instrucciones SQL. Esto ayuda a reducir la cantidad de código, mejorar la organización del proyecto y hacer que el desarrollo sea más rápido.

Además, los ORM suelen incluir protección contra ciertos errores comunes y facilitan la modificación de la estructura de la base de datos mediante migraciones.

Entre las desventajas del ORM se encuentra que puede generar consultas menos eficientes en algunos casos, especialmente cuando se manejan operaciones complejas o grandes cantidades de información. También puede ser necesario conocer su funcionamiento interno para evitar problemas de rendimiento y tener menos control sobre las consultas que realmente se ejecutan en la base de datos.

Por otro lado, SQL permite trabajar directamente con la base de datos, ofreciendo mayor control sobre las consultas y la posibilidad de optimizar el rendimiento. Una de sus principales ventajas es que permite realizar operaciones complejas de manera más precisa y aprovechar las funciones específicas del sistema gestor de base de datos. Sin embargo, como desventaja, requiere escribir manualmente las consultas, lo que puede aumentar la cantidad de código y hacer más difícil el mantenimiento cuando los proyectos crecen.

En conclusión, el ORM es una herramienta práctica que facilita el desarrollo de aplicaciones al simplificar el manejo de datos, mientras que SQL proporciona mayor control y rendimiento. La elección entre ambos dependerá de las necesidades del proyecto, aunque en muchos casos se pueden utilizar ambos para aprovechar las ventajas de cada uno.

¿Por qué Django apuesta por el ORM?

Django utiliza un ORM porque busca facilitar el desarrollo de aplicaciones web permitiendo que los programadores trabajen con la base de datos mediante objetos de Python en lugar de escribir consultas SQL manualmente. Esto hace que el código sea más sencillo de entender, mantener y modificar.

Además, el ORM de Django ayuda a reducir errores, mejora la seguridad al prevenir ciertos ataques como la inyección SQL y permite cambiar de sistema gestor de base de datos con menos cambios en el código. También proporciona herramientas como migraciones para administrar fácilmente los cambios en la estructura de la base de datos.

Aunque en algunos casos SQL directo puede ofrecer más rendimiento y control, el ORM de Django permite desarrollar aplicaciones de manera más rápida y organizada, por lo que es una opción adecuada para la mayoría de proyectos.